

Artificial Intelligence 2 KU

Exercise 1

Group 18

March 26, 2026



Institute of Software Engineering and Artificial Intelligence
Inffeldgasse 16b/II
8010 Graz

Nr.	Name	Matriculation number
1	Laura Maria Höber	12009439
1	Natalia Bocharova	12244471
2	Máté Hipságh	12445931

Contents

1	Exercise 1.a)	2
2	Exercise 1.b)	4
2.1	Results	5
2.1.1	Individual contributions to the result	6
2.2	Discussion	7

1 Exercise 1.a)

The following code shows the implementation in Minizinc, which is supposed to solve a simple constraint satisfaction problem. We start with a given set of variables V containing x_1 and x_2 , each having a domain of possible values $dom(x_i)$ and a set of constraints C containing c_1 and c_2 . The "solve satisfy" command simply searches for one possible solution for the problem, so that all constraints are fulfilled, not using any optimization in the process.

Script

```
%Definition of variables
var 1..4: x1;
var 1..3: x2;
%Definition of constraints
constraint x1>x2;
constraint x2=3;

%Solve problem
solve satisfy;
```

Output

```
x1 = 4;
x2 = 3;
```

```
Finished in 511msec.
```

The detailed procedure that is used to find such a solution is shown in Figure 1. We first apply node consistency for the constraints that apply to only one variable (unary constraints). In this case, the domain of x_2 is reduced to a single value.

Then, via forward tracking, we choose a certain possible value in the domain of x_1 , check if there are any values left in the domain of x_2 to fulfill c_1 , and if that is not the case, backtrack and choose another possible value for x_1 . This is repeated until both domains contain values for which all constraints are satisfied.

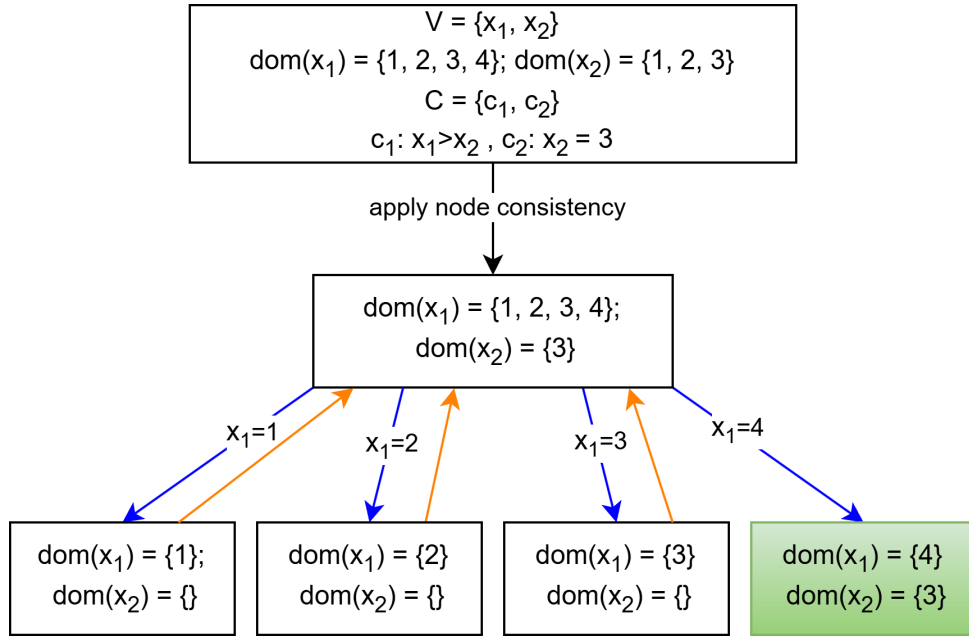


Figure 1: Forward checking and backtracking algorithm

As can be seen by Figure 1, it takes 4 tries to reach a possible result when using the default order of going through the values of x_1 from left to right. It would be more efficient to start with the largest value, according to the "succeed first" principle, which chooses a value that maximizes the number of remaining options. But since the algorithm is complete, an optimal solution will always be found, regardless of the order of search.

2 Exercise 1.b)

The following section explains the details about our approach of a genetica algorithm, which is used to solve the "512 super-queen problem".

Population Generation

One population contains a number of `POPULATION_SIZE`: individuals, where each individual represents one chess board configuration. The chessboard is defined purely by the position of queens, i.e. it is a vector of numbers between 0 and 512, describing the location of a queen in each column. Through permutation it is ensured, that every number only occurs once, i.e. no two queens occupy the same row or column.

Fitness Function

To evaluate how well an individual fulfills the goal, we simply count the number of queens that are not threatened by any other queens on the board, which results in a discrete value between 0 and 512. This is achieved by checking for every queen, if another queen is positioned in the same row, the rising diagonal, the falling diagonal, or the 8 possible positions, where a knight could move. Because the calculation includes many for loops, it was optimized with numba njit.

Selection Method

In each generation, the fitness of each individual is calculated. A certain number of best individuals are taken over unaltered to the next generation. The remaining individuals are necessary to get the full number of `POPULATION_SIZE`: individuals is a mix of these best individuals, combined with a crossover operation and slightly altered through mutation.

Crossover Operator

This operator combines two individuals. First, a random (consecutive) section of columns is selected from the first individual. The remaining columns are filled up with the columns from the other individual. For this, we only use columns from the second individual, where the queen occupies a row that isn't already occupied in the selection of the first individual. This avoids introducing new conflicts.

Mutation Operator

This operator does not randomly change the row position of a queen. In order to keep the queens on separate rows, each mutation chooses two queens and swaps there position. Additionally, to prevent random performance decrease, each mutation is only accepted if it leads to an improvement in fitness. If not, the mutation is reversed and the individual will be taken over unaltered for the following steps.

Termination Condition

Ideally, the algorithm terminates once all queens reached a position, where none of them threaten each other anymore, i.e. the fitness reaches a value of 512. If this is not the case, the algorithm ends once the number of manually chosen `MAX_GENERATIONS` is reached.

Hyperparameters

- **MAX_GENERATIONS**: Maximum number of allowed generations, in case no optimal solution is reached
- **POPULATION_SIZE**: Number of individuals in one generation
- **ADAPT_SIZE**: If true, the **POPULATION** size is doubled after 60%, 80%, and 95% of the queens are already safe
- **BEST_RATIO**: Ratio of best individuals (within an ordered list), that are taken over unaltered to the next generation
- **RAND_RATIO**: Ratio of best individuals (within an ordered list), that are used to generate new mixed individuals through crossover or mutation methods
- **CROSS**: If true, apply crossover operation to create new mixed individuals for the next generation
- **CROSS_DELAY**: Delays the crossover operation until a certain fitness is reached
- **MUTATE**: If true, apply mutation to create new mixed individuals for the next generation
- **MUTATION_NUM**: Number of mutations performed on one individual
- **STAGNATION_DETECTION**: If true, re-initialize a certain part of individuals in the new population by generating random individuals, completely independent from prior selections
- **MAX_STAGNATION**: Maximum number of generations that did not improve the fitness, before part of the population is randomized
- **RESET_RATIO**: Ratio of individuals in the new population that are completely randomized after stagnation was detected

2.1 Results

Figure 2 and Figure 3 show an initial solution that was found within a reasonable number of generations and an acceptable processing time. The red lines mark where random shuffling is applied in case the algorithm stalls, and the dashed blue lines mark where the population size is doubled. The hyperparameters for this solution were set as follows:

Table 1: Hyperparameters for an initial optimal solution.

Hyperparameter	Value	Hyperparameter	Value
MAX_GENERATIONS	10000	BEST_RATIO	0.05
POPULATION_SIZE	100	RAND_RATIO	0.05
ADAPT_SIZE	True	MUTATE	True
STAGNATION_DETECTION	True	MUTATION_NUM	1
MAX_STAGNATION	10	CROSS	True
RESET_RATIO	0.5	CROSS_DELAY	505

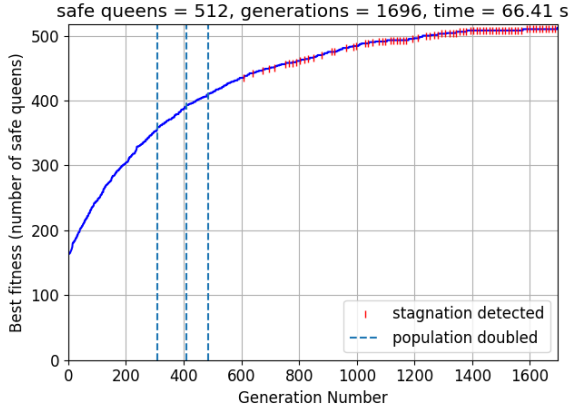


Figure 2: Course of maximum fitness for the final solution.

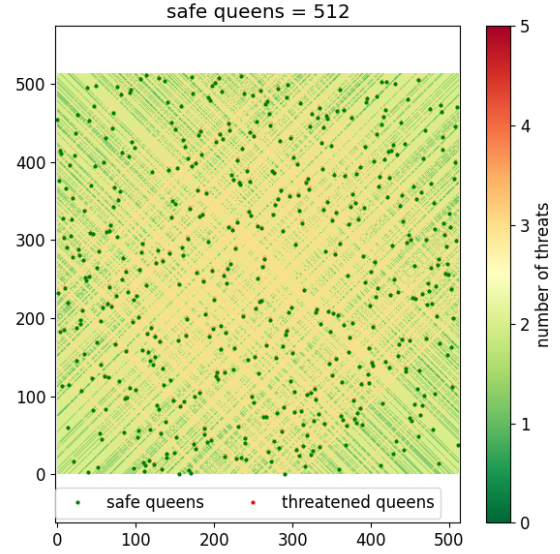


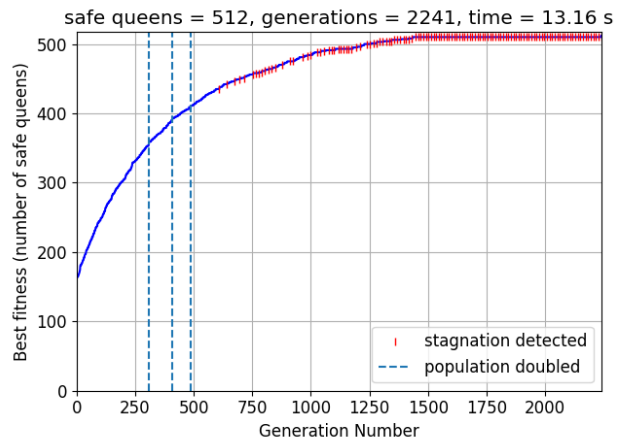
Figure 3: Visualization of the final board arrangement and heatmap of threatened areas.

2.1.1 Individual contributions to the result

The following results show how slightly altering or switching off certain parts of the genetic algorithm influences the result, in order to see how much each hyperparameter contributed to the final outcome. A random seed of 42 was set to prevent random fluctuations from influencing the comparison with the initial result.

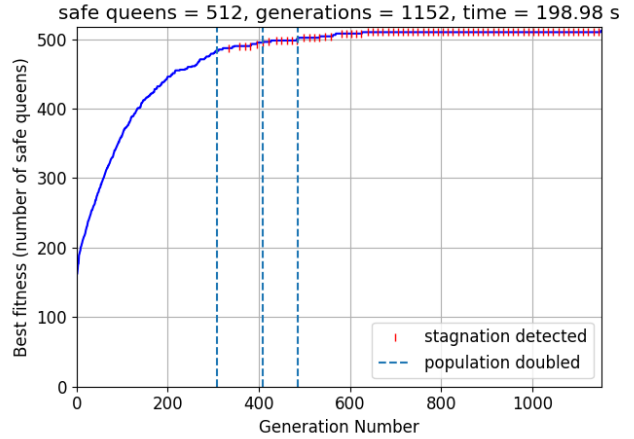
CROSS = False

The algorithm terminates significantly faster without the crossover operation, although it requires about 600 more generations. However, for the final few positions, it struggles to find the optimal configuration for a long time. Therefore, restricting the application of crossover to find the final few optimal positions seems to be a good tradeoff.



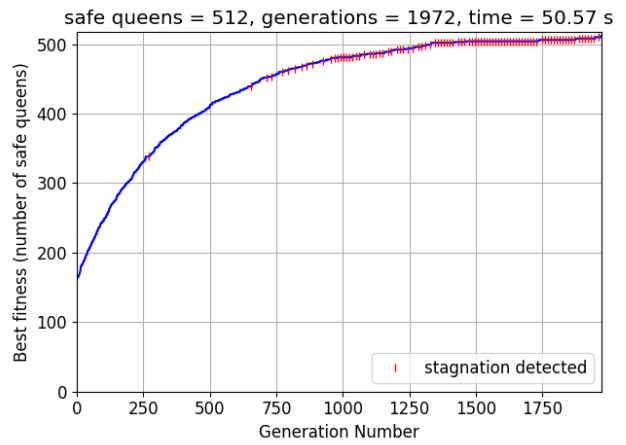
CROSS_DELAY = 0

If the crossover operation is included from the beginning, the total number of necessary generations decreases, but at the cost of tripling the processing time. This further shows that its restricted application to the final iterations is the optimal choice.



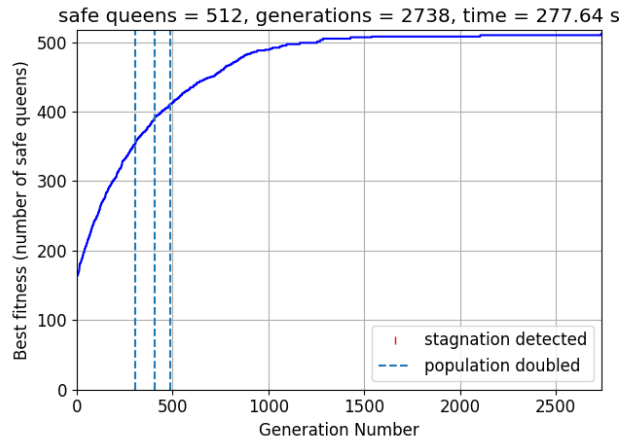
ADAPT_SIZE = False

Adapting the population size does not seem to improve the algorithm. Without it, the algorithm only needs 3 more generations to find the solution, but it is more than 10 seconds faster. Adapting the population size therefore seems redundant.



STAGNATION_DETECTION = False

If no stronger randomization is introduced when the algorithm begins to stagnate around a suboptimal solution, the algorithm takes more than 3 minutes longer to terminate. Strong randomization in the final phase is necessary to avoid stagnation.



2.2 Discussion

Overall, the algorithm already performs well in its initial form, only needing about 1 minute to find an optimal solution with 512 queens positioned in a way that they don't threaten each other. However, the crossover operation does not seem necessary in this case, as the version using only mutation reached the final solution already about 13 seconds, even though it needed a much higher number of generations.